

Documentación: Patrón DTO (Data Transfer Object)

Este documento detalla la investigación, diseño e implementación del patrón DTO en el proyecto **FutManager**.

? ¿Qué es un DTO (Data Transfer Object)?

Un **DTO (Objeto de Transferencia de Datos)** es un patrón de diseño cuyo propósito es transportar datos entre diferentes capas o subsistemas de una aplicación (en nuestro caso, desde el Frontend en React hacia la API REST en Spring Boot, y viceversa).

🌟 Ventajas de Incorporar DTOs en el Proyecto

- **Desacoplamiento:** Separa el modelo de persistencia (Base de Datos / JPA Entities) de la representación en la API REST. Si cambias una columna en la base de datos, no rompes directamente la interfaz del frontend.
- **Seguridad:** Evita la exposición involuntaria de campos internos (como IDs autoincrementales innecesarios, campos de auditoría, contraseñas, etc.).
- **Eficiencia en Red:** Permite empaquetar únicamente los datos necesarios para una vista específica, reduciendo el tamaño del cuerpo JSON en las peticiones HTTP.
- **Prevención de Bucles de Serialización:** En relaciones bidireccionales JPA (como @ManyToOne y @OneToMany), serializar directamente las entidades a JSON genera bucles infinitos (StackOverflowError). Los DTOs aplanan las relaciones evitando este problema.

🔧 Cómo se Incorporan en FutManager (Manteniendo la

Funcionalidad)

Dividimos los DTOs en dos grupos principales:

- **Request DTOs:** Reciben payloads de escritura (POST, PUT). Utilizan IDs simples para vincular relaciones (ej. jugadorId en vez del objeto Jugador completo) facilitando la entrada JSON.
- **Response DTOs:** Envían payloads detallados de lectura (GET). Anidan otros DTOs para estructurar la respuesta y que el frontend muestre nombres de equipos, países, etc.

Código e Implementación de los DTOs en el Backend

1. DTO de Carta FUT (Peticiones de Escritura)

Archivo: CartaFUTRequestDTO.java

```
package com.futmanager.dto;

public class CartaFUTRequestDTO {
    private Long id;
    private Long jugadorId; // ID simple para evitar mandar el objeto completo
    private Long equipoId; // ID de equipo asociado
    private int media;
    private int ritmo;
    private int tiro;
    private int pase;
    private int regate;
    private int defensa;
    private int fisico;
    private String tipoCarta;

    // Constructores, Getters y Setters...
}
```

2. DTO de Carta FUT (Respuestas de Lectura)

Archivo: CartaFUTResponseDTO.java

```
package com.futmanager.dto;

public class CartaFUTResponseDTO {
    private Long id;
    private JugadorDTO jugador; // DTO anidado
    private EquipoDTO equipo;   // DTO anidado
    private int media;
    private int ritmo;
    private int tiro;
    private int pase;
    private int regate;
    private int defensa;
    private int fisico;
    private String tipoCarta;

    // Constructores, Getters y Setters...
}
```

3. DTO de Alineación (Request de Guardado)

Archivo: AlineacionRequestDTO.java

```
package com.futmanager.dto;

import java.util.Map;

public class AlineacionRequestDTO {
    private Long id;
    private String nombre;
    private String formacion;
    private Map<String, Long> posiciones; // Clave -> ID de la carta

    // Constructores, Getters y Setters...
}
```



Clases Mapeadoras (Mappers)

Implementamos clases mapeadoras para traducir entre entidades y DTOs de manera segura y controlada. Ejemplo de `CartaFUTMapper.java`:

```
public class CartaFUTMapper {  
    public static CartaFUTResponseDTO toResponseDTO(CartaFUT carta) {  
        if (carta == null) return null;  
        return new CartaFUTResponseDTO(  
            carta.getId(),  
            JugadorMapper.toDTO(carta.getJugador()),  
            EquipoMapper.toDTO(carta.getEquipo()),  
            carta.getMedia(),  
            carta.getRitmo(),  
            carta.getTiro(),  
            carta.getPase(),  
            carta.getRegate(),  
            carta.getDefensa(),  
            carta.getFisico(),  
            carta.getTipoCarta()  
        );  
    }  
}
```